

Local Evidence RAG

Benchmark locale tra RAG su PDF grezzo e RAG su Markdown
compilato

Report tecnico in stile paper scientifico

Autore: Antonio Cioffi
Repository: rag-vs-markdown
Data progetto: Maggio 2026
Versione report: 11 maggio 2026
Ambito: RAG locale, PDF extraction, Markdown compilation, retrieval evaluation

Abstract

Questo report documenta il progetto rag-vs-markdown, un benchmark locale progettato per valutare se una pipeline di Retrieval-Augmented Generation (RAG) ottiene risposte migliori quando i documenti sorgente PDF vengono prima trasformati in Markdown strutturato rispetto a una pipeline RAG applicata direttamente al testo estratto dai PDF. Il repository risulta descritto come benchmark locale per confrontare RAG su PDF grezzo e RAG su Markdown compilato, con 50 domande, ChromaDB, modello Qwen 0.8B e un evaluation framework. La versione verificata del benchmark include un dataset gold-standard da 50 domande, un report di valutazione generato il 2026-05-11 e una pipeline architetturale basata su PyMuPDF, chunking, embedding con all-MiniLM-L6-v2, ChromaDB, prompt templating e generazione tramite Qwen 3.5 0.8B via Ollama/llama.cpp. Il risultato aggregato disponibile mostra una media di 2.56 punti su 5, pari al 51.2%, con prestazioni migliori sulle domande di estrazione tabellare e peggiori sulle domande negative/di assenza vera. Il progetto è documentato come MVP sperimentale di benchmark locale. Il report disponibile espone un batch aggregato, non una comparazione statistica separata tra tutte le pipeline A/B/C; per questo le conclusioni vengono formulate solo sui dati effettivamente disponibili.

Parole chiave: Retrieval-Augmented Generation, Markdown compilation, PDF parsing, ChromaDB, Qwen, local LLM, benchmark, hallucination evaluation, evidence-grounded QA.

Contents

1	Introduzione	3
2	Repository e stato verificato	3
2.1	Nota di verificabilità	4
3	Obiettivo scientifico	4
3.1	Ipotesi principale	4
3.2	Ipotesi alternativa	4
3.3	Rischio metodologico centrale	4
4	Architettura del sistema	4
4.1	Componenti	5
4.2	Le tre pipeline concettuali	5
5	Corpus documentale	5
6	Dataset gold-standard	6
6.1	Distribuzione per tipo	7
6.2	Perché questa struttura è sensata	7
7	Schema di valutazione	7
8	Risultati quantitativi	8
8.1	Risultato generale	8
8.2	Risultati per tipo di domanda	8
8.3	Risultati per categoria	9
8.4	Distribuzione dei punteggi	9

9	Analisi dei risultati	9
9.1	Punto forte: estrazione tabellare	9
9.2	Punto debole: assenza vera e domande negative	9
9.3	Risultato anomalo: multi-document meglio di simple	9
9.4	Conclusione empirica provvisoria	10
10	Codice e implementazione	10
10.1	Repository hygiene	10
10.2	Schema logico della pipeline	11
10.3	Ruolo del modello Qwen 0.8B	11
11	Confronto con la letteratura	11
12	Limiti osservati	12
13	Conclusione	12
A	Appendice A: punteggi dettagliati	13
B	Appendice B: esempio di riga del dataset gold	13
C	Appendice C: bibliografia essenziale	13

1 Introduzione

Il progetto rag-vs-markdown nasce da una domanda pratica: nei sistemi RAG locali, la qualità delle risposte dipende solo dal modello e dal database vettoriale, oppure dipende in modo forte anche dalla forma documentale data in pasto al sistema? L'ipotesi di lavoro è che un PDF convertito in Markdown strutturato possa conservare meglio gerarchia, titoli, tabelle e concetti rispetto al testo grezzo estratto da PDF, rendendo più semplice il retrieval dei passaggi rilevanti.

La domanda di ricerca è quindi:

Quanto migliora, o peggiora, la qualità delle risposte se trasformo documenti tecnici lunghi in Markdown strutturato prima di indicizzarli, invece di fare RAG direttamente sul testo PDF estratto?

Il progetto è interessante perché sposta il focus da “quale modello usare” a “come preparare la conoscenza”. Questa distinzione è importante: un modello piccolo e locale può fallire non solo perché è debole, ma perché riceve chunk rumorosi, senza struttura e con metadati poveri. In un RAG, la qualità del contesto recuperato è spesso più determinante della qualità stilistica della generazione.

2 Repository e stato verificato

Il repository pubblico è rag-vs-markdown, sotto l'account GitHub cioffiAI. La descrizione GitHub indicava: benchmark locale per confrontare RAG su PDF grezzo vs RAG su Markdown compilato; 50 domande; ChromaDB; Qwen 0.8B; evaluation framework.

Table 1: Stato dei principali elementi verificati nel repository.

Elemento	Stato	Nota tecnica
Repository	Pubblico	cioffiAI/rag-vs-markdown.
Commit principale	Verificato	Branch predefinito: master. Commit 67916bc0...: <i>feat: implement RAG benchmark pipeline with 50 questions, ChromaDB index, and evaluation.</i>
Dataset gold	Verificato	File data/gold_questions.csv, 50 domande annotate.
Report benchmark	Verificato	File reports/benchmark_results.md, generato il 2026-05-11T18:43:32.961671.
File esclusi da Git	Verificato	data/raw/, data/chromadb/, data/extracted/, data/processed/, data/logs/, reports/*.json.
Raw PDF e log JSON	Non versionati	Coerente con una scelta di repository leggero; riduce però la riproducibilità completa da clone pubblico.

2.1 Nota di verificabilità

Il report separa tre livelli:

1. **Dati verificati:** file recuperati dal repository, commit e report benchmark.
2. **Dati ricostruiti dal contesto progettuale:** obiettivo delle tre pipeline e struttura MVP.
3. **Dati non verificabili dal solo repository pubblico:** raw PDF, log JSON completi, output per singola pipeline, configurazioni runtime locali effettive.

Questa distinzione è necessaria: un paper tecnico credibile non deve trasformare un'intenzione progettuale in risultato empirico.

3 Obiettivo scientifico

Il progetto testa una forma di *knowledge compilation*: prima di indicizzare un documento in un RAG, il documento viene pulito e riscritto in un formato più leggibile per il sistema. Nel caso specifico, il formato intermedio è Markdown.

3.1 Ipotesi principale

H1: una rappresentazione Markdown strutturata dei documenti tecnici migliora il retrieval e la fedeltà della risposta rispetto a chunk ottenuti direttamente da testo PDF grezzo.

3.2 Ipotesi alternativa

H0: la conversione in Markdown non produce un miglioramento misurabile, oppure introduce errori di compilazione che compensano il vantaggio strutturale.

3.3 Rischio metodologico centrale

Il rischio principale è confondere tre fenomeni diversi:

1. miglioramento reale del retrieval;
2. miglioramento apparente dovuto a domande più facili o scoring più tollerante;
3. peggioramento nascosto dovuto a perdita di dettagli durante conversione PDF → Markdown.

Per questo il progetto usa 50 domande manuali con categorie diverse, incluse domande negative.

4 Architettura del sistema

Il commit principale contiene la seguente architettura sintetica della pipeline:

```
PDF -> [PyMuPDF] -> Testo/Markdown -> [Chunking] -> [Embedding: all-MiniLM-L6-v2]
                                     |
[Query] -> [ChromaDB (in-memory)] -> Top-K chunks -> [Prompt templating]
                                     |
[Qwen 3.5 0.8B via Ollama/llama.cpp] -> Risposta -> [Logging JSON]
```

4.1 Componenti

Table 2: Componenti principali della pipeline.

Componente	Tecnologia	Funzione
Parsing PDF	PyMuPDF	Estrazione del testo dai PDF tecnici.
Compilazione Markdown	Euristiche/processing locale	Pulizia del testo, preservazione di sezioni, titoli, tabelle e struttura.
Chunking	Pipeline locale	Suddivisione dei documenti in unità recuperabili.
Embedding	all-MiniLM-L6-v2	Trasformazione dei chunk in vettori semantici.
Vector database	ChromaDB	Indicizzazione e retrieval Top-K. Nel commit è indicato come in-memory.
Generazione	Qwen 3.5 0.8B via Ollama/llama.cpp	Generazione della risposta sulla base del contesto recuperato.
Logging	JSON	Registrazione degli output sperimentali; i file JSON sono esclusi da Git.

4.2 Le tre pipeline concettuali

Il disegno sperimentale completo prevedeva tre pipeline:

Table 3: Pipeline sperimentali previste.

Pipeline	Nome	Descrizione
A	PDF diretto	Estrazione del testo dal PDF, chunking, embedding, retrieval e risposta. È la baseline.
B	PDF → Markdown	Estrazione e conversione in Markdown strutturato prima del chunking. Testa il valore della pulizia documentale.
C	Markdown + schede concettuali	Oltre al Markdown, genera schede di concetti, definizioni, limiti e relazioni. Testa una forma più spinta di knowledge compilation.

Il report benchmark verificato espone però un risultato aggregato di batch, non una tabella separata A/B/C. Quindi, nella forma attuale, il materiale verificato documenta l'esecuzione di un benchmark RAG locale e metriche aggregate, senza isolare statisticamente la superiorità della pipeline Markdown.

5 Corpus documentale

Il dataset gold copre 10 documenti PDF, scelti per rappresentare sia letteratura RAG/LLM sia documenti tecnici con tabelle e architetture.

Table 4: Documenti coperti dal benchmark.

ID	Documento e ruolo nel benchmark
01	01_rag_original_lewis_2020.pdf Paper fondativo RAG; domande su RAG-Sequence, RAG-Token, DPR e memoria non parametrica.
02	02_self_rag_asai_2023.pdf Retrieval adattivo e critique tokens; utile per factuality e controllo dei documenti recuperati.
03	03_attention_transformer_vaswani_2017.pdf Architettura Transformer; attenzione, multi-head attention, tabelle BLEU.
04	04_yolov7_wang_2022.pdf Documento tecnico vision-heavy con tabelle e architetture; stress test fuori dal solo NLP.
05	05_llm_evaluation_survey_chang_2023.pdf Survey sulla valutazione LLM; benchmark, safety, LLM-as-judge.
06	06_foundation_models_bommasani_2021.pdf Foundation models, scaling, rischi sistemici, bias, trasferibilità.
07	07_ragas_es_2023.pdf Metriche specifiche RAG: faithfulness, answer relevancy, context precision/recall.
08	08_crag_yang_2024.pdf Benchmark/valutazione RAG real-world; degradazione del retrieval e domini.
09	09_helm_liang_2022.pdf Valutazione olistica: accuracy, calibration, robustness, fairness, bias, toxicity, efficiency.
10	10_tree_of_thoughts_yao_2023_negative_control.pdf Negative control; usato per testare rifiuto, assenza di evidenza e allucinazione.

6 Dataset gold-standard

Il file `data/gold_questions.csv` contiene 50 domande annotate manualmente. Ogni riga include:

- ID domanda;
- tipo;
- categoria;
- documento o documenti attesi;
- domanda;
- risposta attesa;
- punti chiave;
- numero minimo di punti richiesti;
- flag answerable.

6.1 Distribuzione per tipo

Table 5: Distribuzione delle 50 domande per tipo.

Tipo domanda	Numero	Percentuale
Simple factual evidence	15	30%
Local reasoning / same document	10	20%
Multi-document comparison	10	20%
Table extraction / table metric	5	10%
Negative / false premise / true absence	10	20%
Totale	50	100%

6.2 Perché questa struttura è sensata

La struttura copre cinque failure mode diversi:

1. **Domanda semplice:** verifica se il sistema trova un'informazione locale.
2. **Ragionamento locale:** verifica se il sistema combina più passaggi nello stesso documento.
3. **Confronto multi-documento:** verifica retrieval distribuito e sintesi.
4. **Estrazione tabellare:** verifica robustezza su tabelle e metriche.
5. **Domanda negativa:** verifica se il sistema sa non rispondere quando l'evidenza manca.

Le domande negative sono la parte più importante per un RAG serio. Un sistema che risponde sempre, anche quando il documento non contiene la risposta, è pericoloso: produce una risposta leggibile ma non fondata.

7 Schema di valutazione

Il benchmark usa una scala massima di 5 punti per domanda. Nel commit è riportata la seguente logica:

Table 6: Schema di scoring.

Componente	Punteggio	Significato
Correctness della risposta	0-2	Valuta se la risposta contiene i contenuti corretti, con possibile credito parziale.
Documento/evidenza corretta	0-2	Valuta se il sistema recupera il documento atteso. Vale 0 se il documento è sbagliato, anche quando la risposta è corretta per conoscenza pregressa.
Assenza di allucinazione	0-1	Premia risposte senza contenuti inventati.
Domande negative	0 o 5	Risposta corretta se il sistema rifiuta/riconosce l'assenza; 0 se allucina.

Formula sintetica per le domande rispondibili:

$$S_q = A_q + E_q + H_q, \quad S_q \in [0, 5]$$

dove A_q è la correttezza della risposta, E_q è la correttezza dell'evidenza/documento recuperato e H_q è il bonus per assenza di allucinazione.

Per il batch:

$$\bar{S} = \frac{1}{N} \sum_{q=1}^N S_q, \quad \text{Normalized} = \frac{\bar{S}}{5} \cdot 100$$

8 Risultati quantitativi

Il file `reports/benchmark_results.md` riporta 50 domande valutate, media 2.56 su 5 e normalizzazione 51.2%.

8.1 Risultato generale

Table 7: Risultato aggregato del benchmark.

Metrica	Valore
Numero domande	50
Punteggio medio	2.56 / 5
Punteggio normalizzato	51.2%
Mediana	2.75 / 5
Deviazione standard campionaria	1.04
Intervallo di confidenza 95% della media	[2.27, 2.85]
Punteggi ≥ 3	25 / 50
Punteggi ≥ 4	11 / 50
Punteggi < 2	11 / 50

Interpretazione secca: il benchmark funziona, ma il sistema non è ancora maturo. Una media del 51.2% indica che la pipeline produce risposte parzialmente utili, ma non sufficientemente affidabili per un uso tecnico senza revisione umana.

8.2 Risultati per tipo di domanda

Table 8: Risultati per tipo di domanda.

Tipo	Count	Mean	Min	Max
local_reasoning	10	2.55	1.0	4.0
multi_document	10	2.70	1.5	4.0
negative	10	2.25	0.0	3.0
simple	15	2.33	1.0	4.0
table_extraction	5	3.60	3.0	4.0

8.3 Risultati per categoria

Table 9: Risultati per categoria semantica.

Categoria	Count	Mean
comparison	10	2.70
factual_evidence	15	2.33
false_premise	3	2.50
out_of_domain	2	3.00
same_document	10	2.55
table_metric	5	3.60
true_absence	5	1.80

8.4 Distribuzione dei punteggi

Table 10: Distribuzione dei punteggi osservati.

Punteggio	Frequenza	Percentuale
0.0	1	2%
1.0	5	10%
1.5	5	10%
2.0	11	22%
2.5	3	6%
3.0	14	28%
4.0	11	22%

9 Analisi dei risultati

9.1 Punto forte: estrazione tabellare

Le domande `table_extraction` ottengono la media più alta: 3.60 su 5. Questo suggerisce che, almeno nel batch valutato, il sistema riesce meglio quando la risposta attesa è vincolata a un dato preciso. Tuttavia il campione è piccolo: 5 domande. Non bisogna trasformare questo in una conclusione troppo forte.

9.2 Punto debole: assenza vera e domande negative

La categoria `true_absence` ha media 1.80. Questo è il segnale più serio. Un RAG scientifico deve saper dire: “questa informazione non è presente nel contesto recuperato”. Se invece tenta di completare la risposta con conoscenza parametrica o inferenze, il sistema diventa meno affidabile proprio nei casi in cui dovrebbe essere più cauto.

9.3 Risultato anomalo: multi-document meglio di simple

Le domande multi-documento hanno media 2.70, superiore alle domande semplici a 2.33. A prima vista sembra controintuitivo: confrontare più documenti dovrebbe essere più difficile che recuperare un fatto singolo. Le spiegazioni possibili sono tre:

1. le domande multi-documento avevano risposte attese più generali e quindi più facili da ottenere parzialmente;
2. il criterio di scoring premiava sintesi corrette anche senza retrieval perfetto;
3. alcune domande semplici richiedevano dettagli puntuali che il modello piccolo o il retrieval non hanno recuperato.

Questa è una tipica trappola nei benchmark: non tutte le domande dello stesso gruppo hanno pari difficoltà reale.

9.4 Conclusione empirica provvisoria

Il progetto mostra che:

- il benchmark è eseguibile;
- il dataset gold è abbastanza vario per una prima validazione;
- il modello locale piccolo produce risultati parzialmente validi;
- la gestione delle domande negative è ancora insufficiente;
- manca una tabella comparativa separata tra pipeline PDF diretto, Markdown e Markdown+schede.

Quindi il valore principale del progetto non è la tesi “Markdown batte PDF”, ma il framework locale usato per misurare in modo controllato l’impatto della preparazione documentale.

10 Codice e implementazione

10.1 Repository hygiene

Il file `.gitignore` esclude correttamente ambienti virtuali, cache Python, file ambiente e directory dati generate:

```
.venv/  
__pycache__/  
*.pyc  
.env  
data/chromadb/  
data/extracted/  
data/processed/  
data/logs/  
data/raw/  
reports/*.json  
.DS_Store  
stderr.txt  
stdout.txt
```

Questa scelta è tecnicamente sensata per evitare commit pesanti e dati grezzi, ma introduce un limite: chi clona il repository non può riprodurre l’esperimento completo senza i PDF sorgente e senza i log JSON originali.

10.2 Schema logico della pipeline

La pipeline può essere espressa così:

```
def run_benchmark(question_set, documents, mode):
    parsed_docs = extract_with_pymupdf(documents)

    if mode == "pdf_direct":
        corpus = parsed_docs.raw_text
    elif mode == "markdown":
        corpus = compile_to_markdown(parsed_docs)
    elif mode == "markdown_cards":
        corpus = compile_to_markdown(parsed_docs) + build_concept_cards(
            parsed_docs)

    chunks = chunk_documents(corpus)
    embeddings = embed(chunks, model="all-MiniLM-L6-v2")
    index = chromadb.Index(embeddings)

    results = []
    for q in question_set:
        context = index.retrieve(q.text, top_k=K)
        answer = qwen_generate(q.text, context)
        score = evaluate(answer, q.expected_answer, q.expected_documents)
        results.append(score)

    return aggregate(results)
```

Questo non va letto come codice letterale del repository, ma come formalizzazione del comportamento descritto dal commit e dai file verificati.

10.3 Ruolo del modello Qwen 0.8B

La scelta di un modello Qwen 0.8B è coerente con un benchmark locale e leggero. Il vantaggio è la riproducibilità su hardware limitato. Lo svantaggio è che il modello può fallire su:

- sintesi multi-documento lunga;
- rifiuto robusto su domande senza evidenza;
- controllo fine delle citazioni;
- risposte con formule, tabelle e dettagli numerici.

Questo è un punto metodologico importante: un punteggio basso non dimostra automaticamente che il RAG sia progettato male; può anche indicare che il generatore locale scelto è troppo piccolo per alcuni task.

11 Confronto con la letteratura

Il progetto tocca quattro filoni scientifici:

1. **RAG classico:** Lewis et al. introducono la distinzione tra memoria parametrica e non parametrica.
2. **RAG controllato/adattivo:** Self-RAG introduce token di retrieval e critique tokens per valutare documenti e generazione.

3. **Valutazione RAG:** RAGAS misura faithfulness, answer relevancy, context precision e context recall.
4. **Valutazione olistica:** HELM mostra che accuracy, bias, robustness, calibration ed efficiency sono dimensioni diverse, non una sola metrica.

Il contributo pratico di rag-vs-markdown è portare questi concetti in un benchmark piccolo, locale, controllabile e pubblicabile come portfolio.

12 Limiti osservati

1. **Risultati per pipeline non presenti nel report recuperato:** il file di benchmark espone un batch aggregato, senza tabella separata per PDF diretto, Markdown e Markdown+schede.
2. **Raw data non versionati:** data/raw/ è ignorato. La scelta mantiene leggero il repository, ma il clone pubblico non contiene i PDF sorgente.
3. **Log JSON non versionati:** reports/*.json è ignorato. Il repository pubblico non espone quindi l'audit completo risposta-per-risposta.
4. **Campione piccolo:** 50 domande sono coerenti con un MVP, ma non con una validazione statistica ampia.
5. **Valutazione manuale:** lo scoring manuale consente giudizi più fini, ma introduce una quota di soggettività.
6. **Modello piccolo:** Qwen 0.8B è coerente con un'impostazione local-first, ma limita la qualità della generazione su sintesi lunghe, rifiuto delle false premesse e controllo delle citazioni.
7. **Possibile contaminazione parametrica:** alcune risposte possono derivare dalla conoscenza pregressa del modello, non dal documento recuperato. Lo scoring controlla parzialmente questo rischio assegnando peso al documento corretto.

13 Conclusione

rag-vs-markdown documenta un benchmark locale per misurare se la preparazione documentale modifica la qualità di una pipeline RAG. Il punto centrale non è costruire una semplice applicazione "chatta col PDF", ma valutare in modo controllato il ruolo di parsing, chunking, struttura Markdown, retrieval e generazione locale.

Il risultato disponibile, 2.56/5, non rappresenta un successo pieno. La lettura più corretta è: il benchmark è operativo e produce metriche aggregate, ma mostra debolezze nette su assenza vera, rifiuto delle false premesse e controllo dell'evidenza. Questo è coerente con un prototipo locale basato su un modello piccolo e con un dataset di valutazione limitato.

La conclusione rigorosa è:

Il framework locale di valutazione RAG, datato maggio 2026, usa 50 domande annotate, un corpus tecnico multi-documento, scoring a 5 punti e una pipeline basata su PyMuPDF, embedding, ChromaDB e Qwen 0.8B. I dati disponibili documentano l'esecuzione del benchmark e i risultati aggregati; l'ipotesi "Markdown meglio del PDF grezzo" resta trattata come ipotesi di lavoro, non come conclusione isolata statisticamente.

A Appendice A: punteggi dettagliati

```
[4.0, 3.0, 2.0, 3.0, 1.0, 4.0, 2.0, 1.0, 2.0, 1.0,
 2.0, 4.0, 3.0, 1.0, 2.0, 2.0, 4.0, 2.5, 4.0, 3.0,
 2.0, 2.5, 1.0, 1.5, 3.0, 3.0, 2.0, 1.5, 4.0, 2.0,
 4.0, 2.5, 2.0, 2.0, 4.0, 4.0, 4.0, 4.0, 3.0, 3.0,
 1.5, 3.0, 1.5, 0.0, 3.0, 3.0, 3.0, 3.0, 1.5, 3.0]
```

B Appendice B: esempio di riga del dataset gold

```
id,type,category,documents,question,expected_answer,key_points,
min_required_points,answerable
Q001,simple,factual_evidence,01_rag_original_lewis_2020.pdf,
"In RAG, qual è la differenza fondamentale tra RAG-Sequence e RAG-Token?",
"In RAG-Sequence ogni documento genera l'intera sequenza; in RAG-Token ogni
token può usare documenti diversi.",
"RAG-Sequence: un documento per l'intera sequenza; RAG-Token: documento
diverso per ogni token",1,True
```

C Appendice C: bibliografia essenziale

References

- [1] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.
- [2] Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.
- [3] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention Is All You Need. *NeurIPS 2017*.
- [4] Wang, C.-Y., Bochkovskiy, A., and Liao, H.-Y. M. (2022). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors.
- [5] Chang, Y., Wang, X., Wang, J., Wu, Y., Yang, L., Zhu, K., et al. (2023). A Survey on Evaluation of Large Language Models.
- [6] Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., et al. (2021). On the Opportunities and Risks of Foundation Models.
- [7] Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation.
- [8] Yang, S., et al. (2024). CRAG-related benchmark material on retrieval-augmented generation and retrieval quality.
- [9] Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., et al. (2022). Holistic Evaluation of Language Models.
- [10] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models.